

Waveshare ESP32-C6 1.9 inch ST7789 TFT-Display mit Touch Oberfläche im Detail

Dieses Tutorial ist über das Waveshare ESP32-C6 Entwicklungsboard mit 1.9 inch TFT Display, welches eine Auflösung von 170 * 320 Pixeln und eine optionale Touchoberfläche bietet. Obwohl das Board auf der Shopseite (<https://www.waveshare.com/esp32-c6-lcd-1.9.htm>) und im Wiki (<https://docs.waveshare.com/ESP32-C6-LCD-1.9>) beschrieben wird, gibt es doch noch einige Überraschungen bei der Nutzung, die ich Euch ersparen möchte. Selbstverständlich sind alle Beispiele als Code in meinem GitHub Repository hinterlegt, was vielleicht das „inoffizielle“ Produkt Repository werden könnte :-)



Übersicht über die Features des Boards

- ESP32-C6FH8 Prozessor mit 8 MB Flashspeicher und 8 MB PSRAM
- 1.9 inch TFT Display mit ST7789V2 Chip und einer Auflösung von 170 x 320 Pixeln
- optional: Touchoberfläche mit CST816 Chip *1)
- optional: RGB LED WS2812 *1)
- optional: IPEX1 Connector für eine externe Antenne *1)
- Micro SD-Karten Leser
- QMI8658 6-Achsen Bewegungs- (IMU-) Sensor
- TCA9554 8-Kanal GPIO Expander
- JST 1.25 Anschluß für eine 3.7V Lithium Batterie mit integrierter Lade- und Entladeschaltung und Ladekontroll-LED
- Reset und Boot-Taster

*1) Wenn Du die Option mit Touch auswählst, erhältst Du zusätzlich den IPEX1-Anschluß, umgekehrt bei der Wahl „ohne Touch“ ist die RGB LED zusätzlich enthalten.

TFT-Display

Wenn Ihr bereits mit dem Bildschirmtyp ST7789 gearbeitet habt, werdet Ihr auch dieses Display recht einfach in Betrieb nehmen. Die Pins für die SPI Schnittstelle sind dokumentiert und für die ersten Schritte benutze ich die Adafruit ST7735/ST7789 Graphik Bibliothek.

Allerdings sind zwei Besonderheiten zu beachten: um eine hohe Geschwindigkeit zu erhalten (insbesondere wichtig bei vielen Zeichnungsoperationen) solltet Ihr die Hardware-SPI Variante nutzen, die ich so auf die richtigen Pins einrichte:

```
SPI.begin(TFT_SCLK, TFT_MISO, TFT_MOSI, TFT_CS);
```

Die zweite Besonderheit ist die Schaltung der Hintergrundbeleuchtung, denn sie erfolgt über einen MOSFET und ist invers ausgelegt, d.h. `digitalWrite(TFT_BL, LOW)` schaltet das Backlight ein.

Um eine Kompatibilität mit anderen Projekten zu erlauben, stelle ich Euch Programme für die drei Graphikbibliotheken „Adafruit ST7789“, „TFT_eSPI“ und „LovyanGFX“ zur Verfügung. Bitte beachtet, das Ihr für TFT_eSPI eine gepatchte Bibliothek benötigt. Einzelheiten dazu findet Ihr in meinem Tutorial „**Getting Started with an ESP32-C6 Supermini device connected to an ST7789 TFT display**“.

Touch Oberfläche

Waveshare dokumentiert die Pin Belegung auf der Shopseite und ich habe versucht, mit mehreren Bibliotheken für den **CST816** Driver Chip diesen zum Laufen zu bekommen. Dieser Chip wird über die I2C Schnittstelle gesteuert und abgefragt, zusätzlich kommen noch die beiden Leitungen Interrupt/IRQ bzw. Reset/RST zum Einsatz.

Leider funktionierte keine der Bibliotheken und registrierte irgendeinen Touch. Im offiziellen GitHub Repository für das Board fand ich im Beispiel „Arduino/08_LVGL_Test“ (*uint8_t getTouch(uint16_t *x,uint16_t *y)*, *Link: https://github.com/waveshareteam/ESP32-C6-LCD-1.9/blob/1fed15e32977de11afe1f710a0f5e915bfcc35a5/02_Example/Arduino/08_LVGL_Test/FT3168.cpp#L53*) dann einen Hinweis: obwohl die beiden IRQ/RST Pins genannt werden und im Schaltplan auch verzeichnet sind, sind sie physisch nicht mit dem Prozessor verbunden. Auf der Shopseite ist sie auch mit „**NC**“ gekennzeichnet, was aber sehr schnell überlesen werden kann. Warum werden diese Pins explizit genannt wenn sie doch nicht genutzt werden können ?

Damit ist auch klar, warum keine der üblichen Bibliotheken funktioniert hat, denn alle benötigen einen oder beide Pins zur Erkennung eines Touches.

Die Lösung besteht darin, das wir ein bestimmtes Register des CST816 Chips über die I2C Schnittstelle ständig auslesen („polling“), um darüber einen Touch zu erkennen und auch die Koordinaten auf dem Bildschirm erhalten. Auch hier stelle ich ein

Beispielprogramm bereit, allerdings nur für die TFT_eSPI Graphik-Bibliothek (für den Micro-SD-Kartenleser habe ich ein Beispielprogramm für alle drei Graphik-Bibliotheken erstellt, welches auch eine Toucherkennung hat).

Micro-SD-Karten-Leser

Da der ESP32-C6 nur über eine freie SPI Schnittstelle verfügt, werden 4 Pins der SPI Schnittstelle des SD-Karten Lesers mit den Pins des Displays geteilt, lediglich der „Chip Select“ („SD_CS“) Pin erhält eine eigene Pin Nummer. Wie bei allen geteilten Bussystem ist es wichtig, in welcher Reihenfolge die unterschiedlichen Geräte (hier das TFT-Display und der SD-Karten-Leser) initialisiert werden. Ich stelle Euch daher für die drei Graphik-Bibliotheken ein Beispielprogramm bereit, welches 2 Bilder im JPG- und zwei Bilder im BMP-Format auf dem Bildschirm anzeigt. Bei jedem Touch wird das nächste Bild angezeigt. Für Bild im JPG-Format nutze ich die Bibliothek „JPEGDEC JPG Bibliothek“, die Bilder im BMP-Format zeichne ich Pixel für Pixel, was deutlich langsamer als beim JPG-Format geschieht.

QMI8658 Bewegungssensor

Dieser Sensor ist ein **IMU (Inertial Measurement Unit)** - ein räumliches Sensorsystem, das die Bewegung, Ausrichtung und Lage eines Objekts im 3D-Raum misst. Sie dient der präzisen Orientierung und Navigation, oft auch zur Überbrückung, wenn GPS-Signale fehlen. Er misst für die X, Y und Z-Achse jeweils die Beschleunigung und die Drehgeschwindigkeit, daher wird er auch 6-Achsen Sensor genannt. Er ist über die **I2C** Schnittstelle ansprechbar und ich nutze die **QMI8658-Arduino-Library** um die Sensordaten abzurufen. Neben der genannten 6 Werte ist im Chip noch ein Temperaturfühler verbaut, denn Werte wir ebenfalls abrufen können.

Die deutlich professionellere Bibliothek „SensorLib“ habe ich nicht probiert, soll aber auch mit diesem Sensor funktionieren.

Dieser Sensor funktioniert ohne Überraschungen, allerdings hat Waveshare es versäumt, die Position der drei Achsen mitzuteilen. Damit wir von den gleichen Richtungen reden, legen wir das Board im Querformat („Landscape“) flach vor uns auf einen Tisch mit dem USB-Anschluss nach links:

- X-Achse: zeigt von der langen Seite weg nach vorne
- Y-Achse: zeigt von der kurzen Seite weg nach rechts
- Z-Achse: zeigt vom Display nach oben

Die genannten Richtungen ergeben positive (ansteigende) Werte, die entgegen liegende Richtungen ergeben negative bzw. absteigende Werte.

Auf Basis der Adafruit Graphik Bibliothek zeige ich Euch die Funktionsweise.

Batterie Anschluß

Hier könnt Ihr eine 3,7 Volt Lithium Batterie an den zweipoligen JST 1.25 Anschluß anbringen, welche über den USB-C Anschluß geladen wird. Die Polarität entspricht der üblichen Polung, aber Ihr solltet die Polarität Eures Boardes vor dem Anschluß der Batterie selber prüfen, um eine Zerstörung der Batterie und/oder weitergehende Beschädigungen zu vermeiden.

Wenn der Akku geladen wird, leuchtet eine grüne LED neben dem USB-Anschluß auf.

Entladeschutz: Das Board verfügt über keinen Entladeschutz, d.h. Ihr solltet nur Akkus mit eingebautem „Batterie Management System“ (BMS) verwenden, denn ansonsten wird die Batterie ungebremst (tief-) entladen, was der Batterielebensdauer nicht förderlich ist.

Immerhin bietet das Board eine integrierte Spannungsmessung, welche über einen Widerstands-Spannungsteiler mit dem GPIO 0 verbunden ist. Leider ist diese Messung nicht wie bei anderen Boards ein- und ausschaltbar, damit fließt auch bei langer Inaktivität

des Boards (Deep Sleep Phase) ein kleiner aber permanenter Strom durch den Spannungsteiler, der ebenfalls den Akku auf lange Sicht entleert.

Der Spannungsteiler besteht laut Datenblatt aus zwei Widerständen von 200K und 100K, was für eine Drittelung der anliegenden Spannung sorgt: aus 3.7 Volt werden 1.233 Volt am Eingang des Analog Digital Wandlers (ADC). Da die Werte trotz konstanter Spannung stetig wechseln, lese ich 20 Werte aus und mittele das Ergebnis, um eine ruhigere Anzeige zu erreichen.

Die ausgelesenen Werte teile ich durch den „BATTERY_VOLTAGE_DIVIDER 322.7“. Der „krumme“ Wert ergibt sich aus dem rechnerischen 1/3 Teiler (da wir die Rohwerte in Millivolt erhalten, teilen wir durch 333), allerdings hat jedes Bauelement eine Toleranz, welches in der Praxis ein etwas falsches Ergebnis bedeuten würde. Ich habe einfach die reale Spannung mit einem Multimeter gemessen und dann den Teiler so angepasst, das auf dem ESP32-C6 Display und dem Multimeter die gleichen Werte erscheinen (das ist die „Kalibrierung“).

TCA9554 GPIO Extender

Auf diesem Board kommt ein Chip zum Einsatz, den ich bislang bei keinem meiner Boards gefunden habe: ein **TCA9554 8-Kanal GPIO Extender**. Da die SPI und I2C Schnittstellen bereits viele GPIOs fest benutzen, kommt hier der GPIO Extender zum Einsatz. Wieder wird die I2C Schnittstelle genutzt, und mit Hilfe der Adafruit xCA9554 Bibliothek könnt Ihr 8 zusätzliche GPIOs für Eingangs- oder Ausgangs Aufgaben nutzen. Auf dem Pinout des Boards sind sie mit EXIO0 bis 7 bezeichnet. Über die Bibliothek definiert Ihr für jeden Port, ob er als Input- oder Output-Port geschaltet werden soll.

In meinem Beispiel werden auf dem Display 8 rote Schaltflächen angezeigt, und bei einem Touch auf die Schaltflächen wechselt die Schaltfläche ihr Farbe („grün“) und die an den bezeichneten Port angeschlossene LED leuchtet auf bzw. nach einem weiteren Touch geht die LED wieder aus (Adafruit Graphik Bibliothek).

Hat dieses Board auch Nachteile ?

Auf den ersten Blick sieht es wie das vollkommene Board aus, allerdings gibt es eine **Schwachstelle**, nämlich die SPI Schnittstelle. Der ESP32-C6 verfügt über **eine** SPI Schnittstelle, die für die Nutzung des Displays und des Micro SD-Kartenlesers genutzt wird (Bus sharing). Wie Ihr gesehen habt, funktioniert die parallele Nutzung auch ausgezeichnet.

Allerdings führt der Hersteller die SPI-Basis-Pins nicht auf die Breadboard Stiftheiste, so dass Ihr **keine zusätzlichen Geräte mit SPI Schnittstelle** (z.B. einen NFC Karten Leser) **anschließen** könnt. Natürlich gibt es kleine Adapterplatinen, die Ihr in den Leser einführen und darüber die Signale abgreifen könntet, aber das wäre mehr für eine temporäre Bastellösung als eine dauerhafte Nutzung gedacht.

Offizielles GitHub Repository

Immerhin stellt Waveshare ein eigenes Repository speziell für dieses Board an, das für Features ein Beispiel bietet. Neben Arduino IDE gibt es auch Beispiele für ESP-IDF sowie die aufgespielte Firmware als fertiges Binay zum Upload. Ich habe diese Beispiele als nicht lehrreich empfunden, denn sie verteilen sich auf mehrere Dateien und bieten zum Beispiel keine Unterstützung bei der Ansteuerung des Displays oder der Touch-Funktion.

Hinweis: Da dieses Board erst seit März 2026 verkauft wird, sind die Beispiele jetzt auch erst 3 Monate alt – vielleicht werden sie ja noch durch den Hersteller erweitert.

Download Links und Versionsstände der benutzten Software

- Arduino IDE 2.3.8: <https://www.arduino.cc/en/software>
- esp32 Boards 3.3.8 (based on ESP-IDF v5.5.4):
<https://github.com/espressif/arduino-esp32>

- Adafruit ST7735|ST7789 1.11.0:
<https://github.com/adafruit/Adafruit-ST7735-Library>
- Adafruit GFX 1.12.6: <https://github.com/adafruit/Adafruit-GFX-Library>
- TFT_eSPI 2.5.43: https://github.com/Bodmer/TFT_eSPI
(gepatchte Version)
- TFT_eSPI (gepatchte Version) basiert auf 2.5.43:
https://github.com/AndroidCrypto/TFT_eSPI
- LovyanGFX 1.2.21: <https://github.com/lovyan03/LovyanGFX>
- CST816 Bibliothek (ohne Version): selbst erstellt
- SD SD-Card Reader 1.3.0: <https://github.com/arduino-libraries/SD>
(ist in esp32 Boards enthalten)
- JPEGDEC JPG Bibliothek 1.8.4:
<https://github.com/bitbank2/JPEGDEC>
- QMI8658 Arduino Library 1.0.1:
<https://github.com/lahavg/QMI8658-Arduino-Library>
- Adafruit xCA9554 1.0.0:
https://github.com/adafruit/Adafruit_XCA9554
- Waveshare Hersteller Shopseite:
<https://www.waveshare.com/esp32-c6-lcd-1.9.htm>
- Waveshare Hersteller GitHub Repository:
<https://github.com/waveshareteam/ESP32-C6-LCD-1.9>

Source Code der Apps

Du findest alle Codes gesammelt in meinem GitHub Archiv <https://github.com/AndroidCrypto/ESP32-C6-LCD-1.9-inofficial>. Für die gepatchte TFT_eSPI Bibliothek verweise ich auf mein Tutorial **“Getting Started with an ESP32-C6 Supermini device connected to an ST7789 TFT display”**, veröffentlicht hier: <https://medium.com/@androidcrypto/getting-started-with-an-esp32-c6-supermini-device-connected-to-an-st7789-tft-display-76b0809e292f>